

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Below is a complete answer that addresses all five tasks and aligns with the rubric (conceptual understanding, application, analysis, and justification).

AI Search Algorithms for Emergency Medicine Delivery Drones

i. Greedy Best-First Search (GBFS) in this Scenario

How it works:

Greedy Best-First Search selects the path that appears **closest to the destination** based only on the heuristic function **$h(n)$** (e.g., straight-line distance). It ignores the actual travel cost already incurred.

Behavior in the flood scenario:

- The drone always moves toward the destination using the estimated shortest direction.
- It quickly finds a route because it explores fewer paths.
- If a road suddenly becomes blocked, the drone must backtrack or search for another path.
- Since GBFS ignores terrain and weather costs, it may choose dangerous or expensive routes.

Strengths

- Very fast decision-making.
- Uses less memory than many other search algorithms.
- Suitable when emergency response speed is the highest priority.
- Can quickly produce a feasible route.

Weaknesses

- Does **not guarantee the shortest or safest route**.
- Can become trapped in local minima if the heuristic is misleading.
- Ignores movement costs caused by floods, weather, or terrain.
- Frequent route recalculations may occur when paths become blocked.

ii. How A* Handles the Same Situation

How it works:

A* evaluates each node using:

$$\begin{aligned} &[\\ f(n) &= g(n) + h(n) \\ &] \end{aligned}$$

Where:

- **$g(n)$** = actual cost from the start to the current node
- **$h(n)$** = estimated cost from the current node to the goal
- **$f(n)$** = total estimated cost

Behavior in the flood scenario

- The drone considers both the **actual travel cost** (weather, terrain, energy usage) and the **estimated remaining distance**.
- When a road becomes blocked, A* recalculates a new optimal path.
- Dangerous or high-cost routes are naturally avoided because they increase **$g(n)$** .

Advantages

- Finds the optimal route if the heuristic is admissible.
- Balances speed with route quality.
- Considers terrain difficulty and weather conditions.
- Produces safer and more reliable navigation.

Disadvantages

- Slower than Greedy Best-First Search.

- Requires more memory.
- Replanning in a rapidly changing environment increases computation.

iii. Impact of Heuristic Accuracy

The quality of the heuristic significantly affects both algorithms.

Heuristic Accuracy	Greedy Best-First Search	A* Search
Highly accurate	Very fast and often near-optimal	Very fast and optimal
Moderately accurate	Acceptable performance	Good balance of speed and accuracy
Poor/inaccurate	Easily misled into bad paths	Still considers actual costs, reducing errors
Overestimates distance	Poor decisions	May lose optimality
Underestimates distance	May still explore wrong paths	Remains optimal but explores more nodes

Analysis

- **GBFS depends almost entirely on heuristic accuracy.**
- **A*** depends on both heuristic and actual cost, making it more robust.
- Better heuristics reduce search time for both algorithms.

iv. Effect of Dynamic Changes (Blocked Paths)

Flood environments change continuously.

Effect on Greedy Best-First Search

- Initially chooses the nearest-looking route.
- If that route becomes blocked, it may need significant backtracking.
- Frequent environmental changes reduce efficiency.
- Can repeatedly choose unsuitable paths because it ignores actual costs.

Effect on A*

- Detects blocked routes during replanning.
- Computes a new optimal path considering updated information.
- More stable under changing conditions.
- Replanning takes additional computation but improves reliability.

Comparison

Dynamic Change	Greedy Best-First Search	A* Search
Blocked road	Often backtracks	Recalculates optimal route
Weather increases travel cost	Usually ignored	Included in $g(n)$
Frequent updates	Performance decreases	Better adaptation
Reliability	Lower	Higher

v. Recommended Algorithm

Recommended: *A Search**

Justification

Although **Greedy Best-First Search** is faster, emergency medicine delivery requires not only speed but also **safe and reliable navigation**.

A* is more suitable because it:

- Finds cost-effective routes.
- Considers terrain difficulty and weather conditions.
- Adapts better when roads become blocked.
- Produces safer navigation decisions.
- Balances speed with optimality using both actual and estimated costs.

In disaster response, avoiding risky or blocked routes is often more important than simply reaching the destination quickly. A slightly longer but safe route is preferable to a fast route that fails.

If computational resources are limited and immediate decisions are required, Greedy Best-First Search can be used for an initial route. However, A* is the better overall choice because it provides the best balance between **real-time performance, optimality, and safety**.

Conclusion

In a dynamic flood-affected environment, **Greedy Best-First Search** offers rapid route selection but is highly dependent on heuristic accuracy and may choose unsafe or costly paths. A* combines actual travel cost with heuristic estimates, allowing it to adapt more effectively to blocked roads, changing weather, and varying terrain costs. While A* requires more computation, its ability to produce safer, more reliable, and near-optimal routes makes it the most appropriate algorithm for emergency medicine delivery drones operating in uncertain and rapidly changing conditions.

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

Optimization Techniques for Smart City Traffic Signal Control

Problem Formulation

The smart city's traffic signal control can be formulated as an **optimization problem**.

Objective Function

The AI system aims to minimize:

- Total vehicle waiting time
- Fuel consumption
- Traffic congestion

This can be represented as:

Minimize:

$$\begin{aligned} &[\\ f(x) = &\text{Waiting Time} + \text{Fuel Consumption} + \text{Traffic Congestion} \\ &] \end{aligned}$$

where x represents the signal timings at all intersections.

Constraints

- Minimum and maximum green-light duration
- Safety rules for pedestrians and emergency vehicles
- Traffic laws
- Coordination among neighboring intersections

Why Traditional Search is Inefficient

Traditional search algorithms (such as Breadth-First Search or Depth-First Search) are unsuitable because:

- The number of possible signal timing combinations across hundreds of intersections is enormous (state-space explosion).
- Traffic conditions change continuously, making previously explored solutions outdated.
- Searching every possible solution would require excessive computation time.
- Real-time traffic management requires fast decision-making.

Therefore, optimization algorithms are preferred over exhaustive search methods.

i. Hill Climbing in this Scenario

How Hill Climbing Works

Hill Climbing is a **local search optimization algorithm**.

Steps:

1. Start with an initial traffic signal schedule.
2. Slightly modify the timing of one or more traffic lights.
3. Evaluate whether the new schedule reduces congestion.
4. If it improves the objective function, accept the new solution.
5. Repeat until no neighboring solution is better.

Example

Current average waiting time:

- 65 seconds

After increasing one green signal by 10 seconds:

- Waiting time becomes 55 seconds.

Hill Climbing accepts the new schedule because it improves the objective.

Advantages

- Simple to implement.
- Fast decision-making.
- Uses little memory.
- Suitable for continuous optimization.

Limitations

- Can become trapped in local optimum solutions.
- Cannot guarantee the global optimum.
- Sensitive to the initial starting point.
- Performs poorly in highly dynamic environments.

ii. Local Maxima, Plateaus, and Ridges (Real-World Interpretation)

1. Local Maximum (Local Optimum)

A solution that is better than nearby solutions but not the best overall.

Traffic Example

The system finds signal timings that reduce congestion in one district.

However, another timing configuration across the city could reduce congestion much further.

Hill Climbing stops because it cannot find a better neighboring solution.

2. Plateau

A flat region where neighboring solutions have nearly identical performance.

Traffic Example

Changing the green light by ± 2 seconds produces almost no improvement.

The algorithm has no clear direction to continue searching.

Result:

- Slow progress
- Random movement
- Possible early termination

3. Ridge

A narrow path where improvement requires changing multiple variables simultaneously.

Traffic Example

Reducing congestion requires coordinating several adjacent intersections.

Changing only one signal at a time provides no improvement.

Hill Climbing cannot move toward the better solution because each individual change appears ineffective.

Summary Table

Problem	Traffic Interpretation	Effect
Local Maximum	Good but not best traffic timing	Stops early
Plateau	Many signal timings give similar results	Slow or no progress
Ridge	Improvement requires changing multiple signals together	Cannot find better solution

iii. How Simulated Annealing and Genetic Algorithms Overcome These Limitations

A. Simulated Annealing (SA)

How it Works

Simulated Annealing occasionally accepts worse solutions during the early stages of the search.

This allows the algorithm to escape local optima.

As the "temperature" decreases, it gradually becomes more selective and accepts mainly better solutions.

Advantages

- Escapes local maxima.
- Explores more of the search space.
- Works well in dynamic optimization.
- Better chance of finding a near-global optimum.

Traffic Example

The system temporarily accepts a slightly worse signal timing because it may lead to a much better traffic configuration later.

B. Genetic Algorithm (GA)

How it Works

Genetic Algorithms simulate natural evolution.

Each traffic signal schedule is treated as an individual.

The algorithm repeatedly performs:

1. Selection
2. Crossover
3. Mutation
4. Evaluation

The best schedules survive to the next generation.

Advantages

- Searches many solutions simultaneously.
- Escapes local optima.
- Highly scalable.
- Excellent for large optimization problems.
- Easily adapts to changing traffic conditions.

Traffic Example

Thousands of different traffic signal schedules are evaluated simultaneously.

The best-performing schedules are combined to create even better solutions.

Comparison

Feature	Hill Climbing Simulated Annealing Genetic Algorithm		
Escapes Local Optima	No	Yes	Yes
Global Search	No	Partial	Yes
Scalability	Medium	High	Very High
Dynamic Adaptation	Limited	Good	Excellent
Multiple Solutions	No	No	Yes

iv. Recommended Approach

Recommended Algorithm: Genetic Algorithm (GA)

Justification

A smart city contains hundreds of interconnected intersections, creating an extremely large and continuously changing optimization problem.

The **Genetic Algorithm** is the most suitable approach because it:

- Efficiently explores a very large search space.

- Evaluates many traffic signal schedules simultaneously.
- Escapes local optima through crossover and mutation.
- Adapts well to changing traffic conditions.
- Produces high-quality solutions for complex optimization problems.
- Scales effectively as the city expands.

Although **Hill Climbing** is fast, it often becomes trapped in local optima. **Simulated Annealing** improves upon Hill Climbing by escaping local optima, but it still evaluates one solution at a time. In contrast, **Genetic Algorithms** search multiple candidate solutions in parallel, making them more effective for large-scale, dynamic traffic optimization.

Conclusion

Traffic signal optimization in a smart city is a complex optimization problem with a vast and dynamic search space. Traditional search methods are inefficient because they cannot explore the enormous number of possible signal configurations in real time. Hill Climbing provides fast local improvements but suffers from local maxima, plateaus, and ridges. Simulated Annealing addresses these issues by occasionally accepting worse solutions, while Genetic Algorithms use evolutionary principles to explore many candidate solutions simultaneously. Therefore, the **Genetic Algorithm** is the most suitable choice due to its **high scalability, adaptability, and ability to produce high-quality solutions in dynamic traffic environments**.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- Explain why this problem requires an online search agent instead of offline search
- Analyze the challenges of partial observability and dynamic environments
- Describe how the rover builds and updates its internal model
- Compare online search with uninformed and informed search strategies
- Justify the most suitable approach considering uncertainty, adaptability, and safety

Online Search Agent for an Autonomous Mars Rover

i. Why This Problem Requires an Online Search Agent Instead of Offline Search

Offline Search

An **offline search** algorithm assumes that:

- The complete environment is known before the search begins.
- All obstacles and paths are available in advance.
- A full route can be planned before moving.

This approach is **not suitable** for the Mars rover because the rover has **no complete map** of the terrain.

Online Search

An **online search agent** makes decisions while exploring the environment.

It:

- Observes the surroundings using onboard sensors.
- Moves one step at a time.
- Updates its knowledge after each movement.
- Replans whenever new information is discovered.

Why Online Search is Needed

The Mars rover:

- Explores unknown terrain.
- Has limited sensing capability.
- Encounters unexpected hazards such as rocks and craters.
- Must make immediate navigation decisions.
- Learns about the environment as it moves.

Therefore, an **online search agent** is the most appropriate approach because it can operate effectively in an unknown and changing environment.

ii. Challenges of Partial Observability and Dynamic Environments

A. Partial Observability

Partial observability means the rover cannot see the entire environment.

Challenges

- Sensors detect only nearby obstacles.
- Hidden hazards may exist beyond the sensing range.
- The rover cannot determine the complete path to the destination.
- Decisions must be made with incomplete information.

Example

A safe-looking path may later reveal a large crater that forces the rover to change direction.

B. Dynamic Environment

Although the Martian landscape changes slowly, the rover's available information changes continuously as it explores.

Challenges

- Newly detected obstacles require route changes.
- Terrain may be rougher than expected.
- Wheel slippage or dust can affect movement.
- The rover must continuously update its navigation plan.

Effects

- Frequent replanning.
- Increased computational effort.
- Need for adaptive decision-making.
- Greater emphasis on safety.

iii. How the Rover Builds and Updates Its Internal Model

The rover maintains an **internal model** of the environment.

Step 1: Initial Knowledge

The rover begins with:

- Landing location
- Mission objective
- Little or no terrain information

Step 2: Sense the Environment

Using cameras, lidar, radar, and other sensors, the rover collects information about:

- Rocks
- Craters
- Slopes
- Safe paths

Step 3: Update the Map

After each movement, the rover:

- Marks safe areas.
- Records hazardous regions.
- Stores explored paths.
- Identifies unexplored locations.

Step 4: Replan

When new hazards are detected, the rover:

- Updates its internal map.
- Computes a safer route.
- Continues exploration.

Continuous Cycle

Sense → Update Map → Plan → Move → Repeat

This allows the rover to improve its knowledge and navigation as it explores.

iv. Comparison of Online Search with Uninformed and Informed Search

Feature	Online Search	Uninformed Search	Informed Search
Environment Knowledge	Unknown	Known	Known
Uses Heuristic	Sometimes	No	Yes
Planning	During movement	Before movement	Before movement
Adaptability	High	Low	Moderate
Handles Obstacles	New Yes	Poorly	Better than uninformed but limited
Suitable for Mars Rover	Excellent	Poor	Limited

Analysis

Uninformed Search

- Does not use heuristic information.
- Explores many unnecessary states.
- Requires a known environment.
- Inefficient for Mars exploration.

Informed Search

- Uses heuristic estimates to improve efficiency.
- Works well when a map is available.
- Cannot fully handle unknown environments without frequent replanning.

Online Search

- Makes decisions while exploring.
- Learns from experience.
- Adapts to newly discovered hazards.
- Best suited for unknown terrain.

v. Recommended Approach

Recommended Approach: Online Search Agent

Justification

The Mars rover operates in an uncertain environment where it has incomplete information and limited sensing capabilities. An **online search agent** is the most suitable approach because it:

- Makes decisions using current sensor data.
- Continuously updates its internal map.
- Adapts to newly discovered hazards.
- Safely avoids obstacles.
- Supports autonomous operation without requiring a complete map.
- Responds effectively to unexpected situations.

Although uninformed and informed search algorithms perform well in environments with known maps, they are less effective when the environment is unknown. The online search agent provides the flexibility and adaptability needed for real-time exploration while prioritizing safety and mission success.

Conclusion

Mars exploration presents an uncertain environment where complete information is unavailable before the mission begins. As a result, **offline search techniques are unsuitable** because they rely on a fully known map. An **online search agent** enables the rover to sense its surroundings, update its internal model, and replan its route as new hazards are discovered. While uninformed and informed search algorithms are effective in known environments, the **online search approach** is the best choice for Mars exploration due to its **ability to handle uncertainty, adapt to changing information, and ensure safe autonomous navigation**.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

AI-Based University Exam Timetable Generation Using Constraint Satisfaction Problems (CSP)

Problem Formulation as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) consists of:

- Variables
- Domains
- Constraints

The goal is to assign values to all variables such that all constraints are satisfied.

In this scenario, the AI system must assign exam times and halls while ensuring that no scheduling conflicts occur.

i. Variables, Domains, and Constraints

1. Variables

Variables represent the exams that need to be scheduled.

Examples:

- Mathematics Exam
- Physics Exam
- Computer Science Exam
- English Exam

Each exam is treated as one variable.

Example:

- X_1 = Mathematics
- X_2 = Physics
- X_3 = Computer Science
- X_4 = English

2. Domains

The domain is the set of possible values that each variable can take.

For each exam, the domain includes:

- Available time slots
- Available exam halls

Example:

Mathematics:

{Monday Morning–Hall A, Monday Afternoon–Hall B, Tuesday Morning–Hall C}

Each variable can only be assigned one valid time slot and hall.

3. Constraints

The AI system must satisfy several constraints.

A. No Student Exam Overlap

A student enrolled in two subjects cannot have both exams at the same time.

Example:

If a student takes Mathematics and Physics:

Mathematics $\neq$ Physics Time Slot

B. Limited Exam Halls

Only a limited number of halls are available.

No two exams can use the same hall at the same time.

C. Subject Ordering

Certain exams must occur before others.

Example:

Programming Lab must be scheduled before the Project Evaluation.

D. Faculty Availability

An exam can only be scheduled when the assigned faculty member is available.

E. Special Accommodation

Students requiring extra support (such as additional time or accessible rooms) must be assigned suitable halls and schedules.

Summary

CSP Component	Example
Variables	Exams (Mathematics, Physics, etc.)
Domains	Available time slots and exam halls
Constraints	No overlap, hall capacity, faculty availability, subject order, special accommodations

ii. How Backtracking Search Works

Backtracking is a systematic search algorithm used to solve CSPs.

Steps

Step 1

Assign the first exam to an available time slot.

↓

Step 2

Assign the second exam.

↓

Step 3

Check whether any constraint is violated.

- If no violation occurs, continue.
- If a violation occurs, undo (backtrack) the previous assignment.

↓

Step 4

Try another available time slot.

↓

Repeat until every exam is scheduled successfully.

Example

Schedule:

- Mathematics → Monday Morning ✓
- Physics → Monday Morning ✗

Conflict:

A student is enrolled in both subjects.

Backtracking removes Physics from Monday Morning and tries:

- Physics → Monday Afternoon ✓

The algorithm then proceeds to schedule the remaining exams.

Advantages

- Guarantees finding a valid timetable if one exists.
- Systematically explores possible schedules.
- Simple and widely used for CSPs.

Disadvantages

- May explore many possibilities.
- Becomes slower as the number of courses increases.

iii. How Constraint Propagation (Forward Checking) Improves Efficiency

Constraint propagation reduces unnecessary searching by eliminating invalid choices early.

Forward Checking

Whenever an exam is assigned:

- Remove conflicting time slots from the domains of related exams.
- Detect impossible schedules immediately.

Example

Suppose:

Mathematics → Monday Morning

Forward Checking automatically removes Monday Morning from the domains of:

- Physics
- Chemistry

because the same students are enrolled.

Instead of discovering the conflict later, it is prevented immediately.

Benefits

- Reduces the search space.
- Detects conflicts early.
- Requires fewer backtracking steps.
- Improves execution speed.
- Handles larger scheduling problems more efficiently.

Comparison

Without Forward Checking	With Forward Checking
More trial-and-error	Early conflict detection
Many unnecessary assignments	Fewer invalid assignments
More backtracking	Less backtracking
Slower execution	Faster execution

iv. Real-World Challenges and Best Strategy for Scalability

Real-World Challenges

Large Number of Courses

A university may have hundreds of courses, greatly increasing the number of possible schedules.

Large Student Population

Thousands of students create many overlapping subject combinations that must be considered.

Limited Resources

- Limited exam halls
- Limited invigilators
- Limited time slots

These increase scheduling complexity.

Dynamic Changes

Unexpected events may require timetable adjustments, such as:

- Faculty unavailability
- Hall maintenance
- Emergency schedule changes
- New student registrations

Multiple Constraints

The AI must satisfy all constraints simultaneously without conflicts, making the problem computationally complex.

Best Strategy for Scalability

The most effective approach is to combine:

- Backtracking Search
- Constraint Propagation (Forward Checking)

Justification

This combination:

- Reduces unnecessary searches.
- Detects conflicts before they occur.
- Handles large scheduling problems efficiently.
- Produces valid timetables faster than using backtracking alone.
- Scales well as the number of courses, students, and constraints increases.

Forward Checking significantly reduces the search space, while Backtracking guarantees that a valid solution will be found if one exists.

Conclusion

Generating a university exam timetable is a classic Constraint Satisfaction Problem (CSP) in which exams are represented as variables, available time slots and halls form the domains, and scheduling requirements act as constraints. Backtracking Search systematically assigns exams while revising decisions when conflicts arise. Constraint Propagation, particularly Forward Checking, improves efficiency by removing invalid choices early, reducing unnecessary backtracking. For large universities with many courses, students, and scheduling constraints, the combination of Backtracking and Forward Checking provides the most scalable, efficient, and reliable solution for producing conflict-free exam timetables.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

Strategic Game AI for Real-Time Decision Making Using Minimax and Alpha-Beta Pruning

i. How the Minimax Algorithm Models This Problem

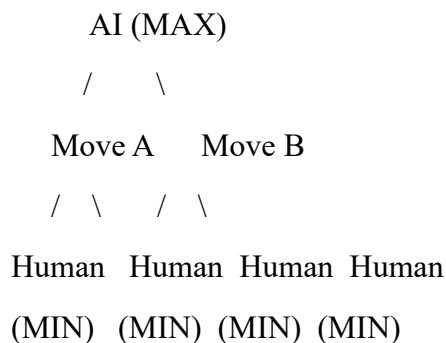
The Minimax algorithm is an adversarial search algorithm used in two-player games where one player tries to maximize the score while the opponent tries to minimize it.

How Minimax Works

The game is represented as a game tree.

- Each node represents a game state.
- The AI (MAX player) chooses the move that maximizes its chances of winning.
- The human player (MIN player) chooses moves that minimize the AI's chances of winning.
- The algorithm explores possible future moves and selects the action with the highest utility value.

Example



The AI assumes the opponent always plays the best possible move and chooses the move with the best worst-case outcome.

Advantages

- Produces rational decisions.
- Considers future consequences of moves.
- Guarantees the optimal move if the entire game tree is explored.

Limitation

- Computationally expensive for games with many possible moves.

ii. Computational Challenges of Large Game Trees

In real-time strategy games, each game state may have many possible actions.

Challenges

A. Large Branching Factor

Each unit can perform multiple actions, resulting in hundreds or thousands of possible moves.

Example:

- 20 possible moves per turn
- Search depth = 6

Number of nodes:

$$\begin{aligned} &[\\ &20^6 = 64,000,000 \\ &] \end{aligned}$$

Millions of game states must be evaluated.

B. Time Constraints

Players expect immediate responses.

The AI cannot spend several minutes searching every possibility.

C. Memory Requirements

Storing large game trees requires substantial memory.

D. Dynamic Game Environment

Opponent actions continuously change the game state.

The AI must frequently recompute its strategy.

Result

Without optimization, Minimax becomes too slow for real-time games.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta Pruning is an optimization technique for Minimax.

It eliminates branches that cannot influence the final decision.

Alpha (α)

The best value that the MAX player can guarantee so far.

Beta (β)

The best value that the MIN player can guarantee so far.

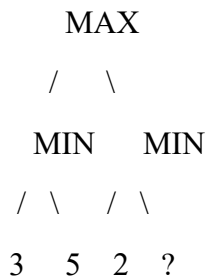
Pruning Rule

If:

$$\begin{aligned} &[\\ &\alpha \geq \beta \\ &] \end{aligned}$$

the remaining branches are ignored because they cannot improve the final decision.

Example



Suppose:

- Left branch returns 3.
- MAX already has $\alpha = 3$.
- Right MIN node finds a value of 2.

Since MIN will never allow a value greater than 2, exploring the remaining branch cannot improve the result.

The remaining branch is pruned.

Benefits

- Evaluates far fewer nodes.
- Produces the same optimal decision as Minimax.
- Reduces computation time.
- Enables deeper searches within the same time limit.
- Improves performance in real-time applications.

Comparison

Feature	Minimax	Alpha-Beta Pruning
Finds Optimal Move	Yes	Yes
Nodes Evaluated	Very High	Much Lower
Search Speed	Slower	Faster
Memory Usage	High	Lower
Suitable for Real-Time Games	Limited	Excellent

iv. Evaluation Function and Justification

Since searching the complete game tree is impractical, the AI uses an evaluation function to estimate the quality of a game state.

Example Evaluation Function

$$\begin{aligned} &[\\ \text{Score} &= 5M + 4R + 3T + 2H - 4E \\ &] \end{aligned}$$

Where:

- M = Military strength (number and strength of units)
- R = Resources available (gold, food, energy)
- T = Territorial control (strategic locations occupied)
- H = Health of units and structures
- E = Enemy advantage (enemy strength and control)

Higher scores indicate more favorable positions for the AI.

Factors Considered

Military Strength

- Stronger armies improve the chance of winning.

Resources

- More resources allow the production of additional units and buildings.

Territorial Control

- Controlling key locations provides strategic advantages.

Health

- Healthier units and structures increase combat effectiveness.

Enemy Advantage

- A stronger enemy position reduces the evaluation score.

Justification

These factors capture the most important aspects of a real-time strategy game. The evaluation function enables the AI to quickly estimate the value of non-terminal game states and make informed decisions without exploring the entire game tree.

v. Recommended Strategy for Balancing Optimality and Real-Time Performance

Recommended Strategy

The best approach is to combine:

- Minimax Algorithm
- Alpha-Beta Pruning
- Depth-Limited Search
- Evaluation Function

Justification

This combination:

- Maintains near-optimal decision-making.
- Greatly reduces computation by pruning unnecessary branches.
- Uses a depth limit to ensure decisions are made within strict time limits.
- Employs the evaluation function to estimate the value of positions when the search reaches the depth limit.
- Adapts well to complex and dynamic game environments.

This strategy provides an effective balance between decision quality and real-time performance, making it suitable for modern strategy games.

Conclusion

The Minimax algorithm models the game as a competition between two rational players, allowing the AI to select moves that maximize its chances of success while anticipating the opponent's best responses. However, the enormous size of the game tree makes exhaustive search computationally expensive. Alpha-Beta Pruning addresses this issue by eliminating branches that cannot affect the final decision, significantly reducing the number of evaluated states while preserving optimality. By combining Minimax, Alpha-Beta Pruning, a depth-limited search, and a well-designed evaluation function that considers military strength, resources, territorial control, unit health, and enemy advantage, the AI can achieve an excellent balance between optimality, scalability, and real-time decision-making, making it the most suitable approach for a competitive real-time strategy game.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand